

TECHNICAL AND BUSINESS DOCUMENTATION

Interntion CRM

A complete guide to the Odoo CRM for managing student internship placements, corporate partnerships, mentoring, support, catalogue services, and operational accountability.

Prepared from the live local implementation

Workspace: `Internsion CRM`

Module: `addons/interntion_crm`

Document date: 15 September 2026

How to Read This Document

This document starts with the business problem, then explains the system architecture, screens, data models, code behavior, and operations. It is written for a reader who is new to Odoo and to this project.

Core idea: Interntion is not using CRM only to collect sales leads. Each student application becomes an operational case that moves from application to screening, matching, interview, offer, active placement, and completion. Employers form a second pipeline. Everything else supports those two journeys.

Contents

1. Business purpose and scope
2. System architecture and why each layer is required
3. Live dashboard walkthrough
4. Student and partner workflows
5. Data models and code rationale
6. Views, menus, actions, and user experience
7. Security and access control
8. Automation, SLA, support, and quotations
9. Seed data and website integration
10. Running, updating, and production guidance

1. Business Purpose and Scope

Interntion helps students access paid internships and career support. The CRM converts that real service journey into a controlled process. Without it, applications, employer conversations, CV reviews, mentor assignments, and urgent support requests would be scattered across spreadsheets, email, and chat.

Business need	CRM capability	Why it is required
Keep every student application visible	Extended <code>crm.lead</code> placement pipeline	Provides one owner, one status, and an auditable journey for each student.
Match students to real opportunities	Internship and employer catalogues linked to applications	Prevents duplicate data entry and makes capacity, employer, sector, and placement status visible.
Respond quickly to students and partners	SLA deadlines, activities, and escalation monitor	Protects service quality: student matching is tracked over seven days; partner response is tracked over 24 hours.
Make management decisions quickly	Live KPI dashboard with drill-down actions	Every dashboard number opens the underlying records instead of becoming a static report.
Handle operational issues	Placement Support tickets	Records student, partner, or mentor issues with priority, owner, state, discussion history, and follow-up activities.

The module is declared as an installable Odoo Sales/CRM application in `addons/interntion_crm/__manifest__.py`. Its dependencies deliberately reuse Odoo capabilities rather than rebuild them: `crm` provides leads and pipelines, `mail` provides chatter and activities, `sale_management` provides quotations, `product` supports sellable add-ons, and `contacts` supports contact relationships.

2. Architecture from Scratch

What Odoo contributes

Odoo is an ERP platform. A custom Odoo module does not need to write login, forms, database persistence, permissions, filters, messaging, or navigation from zero. Python model classes define business objects and behavior; XML defines screens, actions, menus, security, and seed records; PostgreSQL stores the data.

Browser → Odoo web application → Interntion CRM module → PostgreSQL database

Docker Compose

`docker-compose.yml` starts repeatable containers for Odoo and PostgreSQL. Docker is required here because it gives every developer the same server and database versions without installing them directly on Windows.

Component	What it does	Why it is present
<code>db</code> service	Runs PostgreSQL 15 with a persistent named volume.	Odoo requires a relational database. The volume keeps CRM records when the container restarts.
<code>odoo</code> service	Runs Odoo, exposes port 8069, and mounts <code>./addons</code> .	The mount lets Python/XML changes be available to Odoo without rebuilding the application image.
Health check and <code>depends_on</code>	Waits for PostgreSQL readiness before Odoo starts.	Avoids Odoo booting while the database is unavailable.
Optional <code>pgadmin</code>	Database administration UI on port 5050.	Useful for authorised diagnostics; it is excluded unless the <code>tools</code> profile is started.
<code>odoo.conf</code>	Configures add-on paths, database host, credentials, logging, and proxy behavior.	Separates environment configuration from business code.

Version note: the module manifest declares version `19.0.2` and the Compose image is specified as `odoo:19.0`. The most recent local Odoo server log identified itself as `17.0-20260630`. Before production rollout, verify the image actually running with `docker compose images` and pin the intended Odoo 19 image digest. This is an environment verification point, not a change made by this document.

3. Live Dashboard Walkthrough

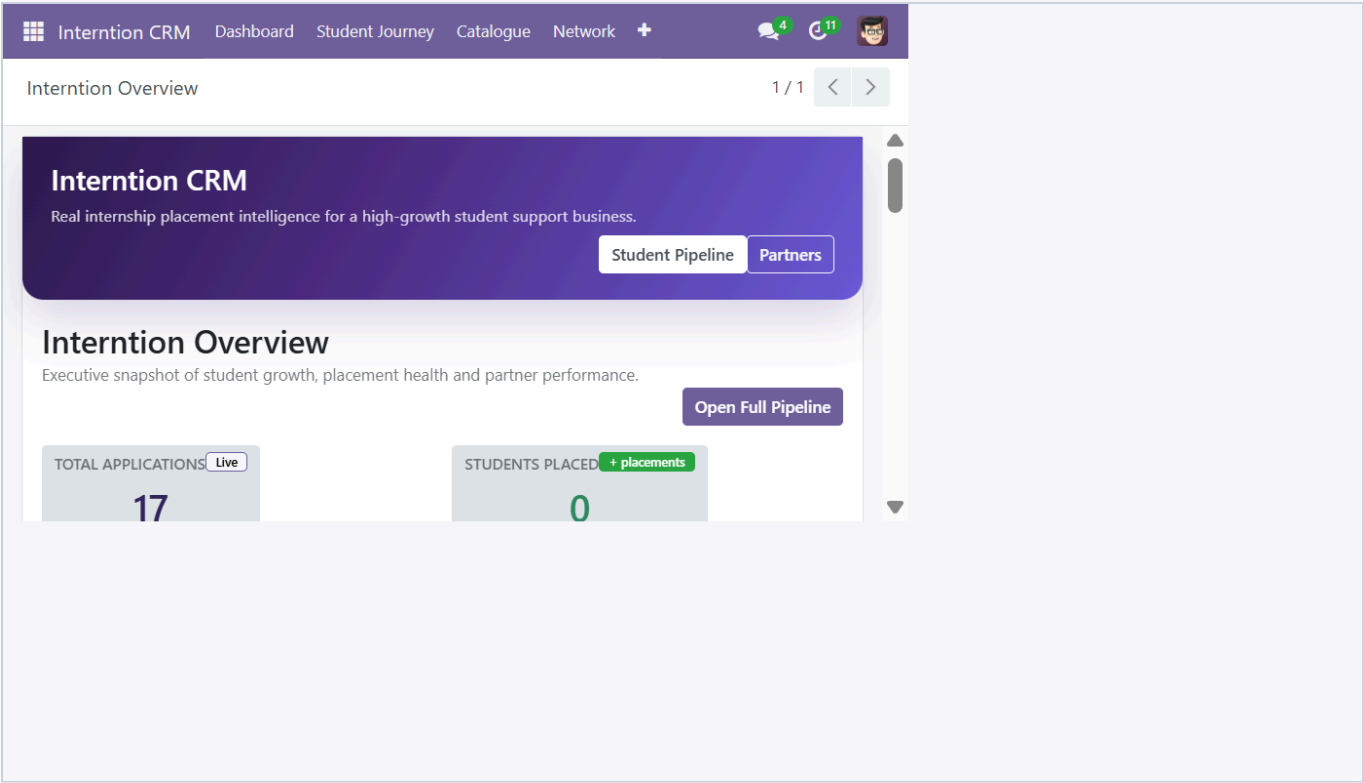


Figure 1. Live Interntion CRM dashboard captured from the local Odoo application.

Screen area	What it shows	Why it was implemented
Top navigation	Dashboard, Student Journey, Catalogue, Network, and support access.	Groups daily work by purpose so staff can navigate with fewer decisions.
Header actions	Student Pipeline and Partners .	Gives executives direct access to the two primary operational processes.
KPI cards	Total applications, placed students, success rate, industry partners.	Shows the health of demand, outcomes, conversion, and supply network at a glance.
Application funnel	New applications through placed cases.	Identifies bottlenecks. Each stage is a button calling a native Odoo action with a stage-specific domain.
Growth health	Open roles, active mentors, at-risk cases, and student SLA rate.	Shows operational capacity and potential service failures, not only sales volume.
Help & support	Open and urgent support-ticket counts plus Request Help .	Makes escalations available from the command screen; it opens the existing support-ticket form.

Drill-down design: dashboard buttons return `ir.actions.act_window` dictionaries. Odoo then opens a filtered kanban/list/form view of real records. This is preferable to a static dashboard because an operational user can move from a warning or metric directly to the cases needing action.

4. The Two Primary Workflows

Student placement workflow

New Application → CV/Profile Review → Matched to Partner → CV & Interview Prep → Interview Scheduled → Offer Extended → Placement Active → Placement Completed

The stage records in `data/crm_stage_data.xml` belong to the Student Placements sales team. Stages make the kanban board useful: a card moves across columns as the real-world work moves forward. **Placement Completed** is marked as won and folded because it is a completed outcome, while **Not Progressing** is folded because it is a closed non-success path.

Important transition	Code behavior	Reason
Lead enters the system	<code>create()</code> schedules an initial SLA activity.	Every case immediately has an accountable follow-up instead of relying on memory.
Matched to Partner	<code>write()</code> records <code>matched_at</code> and <code>sla_completed_at</code> .	Creates a measurable point at which the student matching commitment was fulfilled.
Placement Completed	<code>write()</code> schedules an "Issue completion certificate" activity.	Ensures the final service obligation is not lost after a successful placement.

Corporate partner workflow

New Enquiry → Qualification Call → Needs/Brief Confirmed → Candidate Shortlist Sent → Agreement/Contract → Active Partner

Partners use a separate team and `interntion_side = 'partner'`. This distinction is necessary because a student requires career/placement work, while a partner requires relationship, brief, contract, and account-management work. Partner leads receive a 24-hour response SLA because early response strongly affects partner confidence.

5. Data Models and Code Rationale

In Odoo, a model is a Python class that maps to a database table. Fields define what the business needs to store. Relations such as `Many2one` create a link to one related record, while `One2many` provides the reverse collection. The model files are imported by `models/__init__.py`, which is required so Odoo can register them at startup.

5.1 The central CRM lead

`models/crm_lead.py` extends, rather than replaces, Odoo’s standard `crm.lead`. This is the correct approach because it retains Odoo features such as assignment, activities, messages, stage history, reporting, and conversion behavior.

Field group	Examples	Why it exists
Student profile	<code>x_course_stream</code> , <code>x_industry_sector</code> , <code>x_application_source</code>	Supports matching, source analysis, and counsellor context.
Placement links	<code>x_internship_id</code> , computed <code>x_employer_id</code> , <code>x_mentor_id</code>	Creates one connected record from candidate to role, employer, and mentor.
Readiness evidence	<code>x_cv_file</code> , <code>x_cv_reviewed</code> , <code>x_mock_interview_done</code>	Turns preparation work into visible checkpoints rather than unrecorded conversations.
Payment/outcome	<code>x_is_paid_internship</code> , <code>x_certificate_issued</code>	Tracks the paid-placement promise and final completion deliverable.

The internship onchange copies its sector and paid status into the application. This reduces manual entry and prevents a counsellor from accidentally classifying an application differently from the chosen internship.

5.2 Operations extension and support

`models/operations.py` further extends `crm.lead` and defines the supporting reference models.

Model / behavior	Implementation	Why it is required
<code>interntion.support.ticket</code>	Name, linked lead, reporter type, HTML description, priority, state, assignee; inherits <code>mail.thread</code> and <code>mail.activity.mixin</code> .	Separates placement incidents from general CRM notes while retaining a clear owner and escalation history.
<code>interntion.industry</code>	Reference model with unique indexed code and sequence.	Maintains a controlled, reusable list for target industries and reporting.
<code>interntion.addon.service</code>	Links an add-on to Odoo <code>product.product</code> .	Lets a lead select additional paid services that can be converted into a real quotation.
<code>_compute_sla_deadline()</code>	Student: creation + 7 days. Partner: creation + 24 hours.	Encodes the service promise consistently for all cases.
<code>_compute_sla_status()</code>	Returns on-track, at-risk, or breached.	Surfaces time-sensitive cases before the deadline is missed.
<code>action_create_addon_quotation()</code>	Creates <code>sale.order</code> and lines from chosen add-ons.	Connects service interest to Odoo Sales instead of retyping charges manually.

5.3 Employer, internship, and mentor models

File and model	Important fields / methods	Business reason
employer.py <code>interntion.employer</code>	Sector, contact information, country, account manager, logo, active partner flag, related internships/application counts; actions open its linked records.	Creates one source of truth for the companies supplying placement opportunities.
internship.py <code>interntion.internship</code>	Employer, sector, description, duration, location, paid flag, stipend, positions, application/placed counts.	Represents the actual role a student can be matched to; count fields make role demand visible.
mentor.py <code>interntion.mentor</code>	User, contact details, expertise, photo, active flag, related application count.	Allows workload and specialist availability to be managed rather than assigned informally.

These models inherit Odoo's `mail.thread` and `mail.activity.mixin`. The first provides chatter/audit messages; the second provides activities and due dates. They are deliberately reused instead of custom messaging code.

5.4 Catalogue and content models

File	Purpose	Why separate from the pipeline
project.py	Project Assistance catalogue: academic level, category, technology stack, price, deliverables.	Projects are offers/products, not individual student cases.
service.py	Four core services with code, description, icon, sequence, website URL, active state.	Maintains consistent service content independent of ongoing opportunities.
testimonial.py	Student name, course, role, quote, image, rating, sequence.	Stores curated marketing proof with ordering and publication control.
faq.py	Question, HTML answer, category, sequence, active state.	Turns frequently repeated queries into reusable knowledge content.

5.5 Dashboard model

models/dashboard.py uses a lightweight `interntion.dashboard` record to compute management metrics from current business data. It is read-only at the view layer. Its helper `_student_stage_domain()` builds an Odoo search domain for one or more stage names, allowing one source of filtering logic to serve several funnel buttons.

`_compute_kpis()` calls `search_count()` for live counts, calculates success rate as $\text{placed} \div \text{total} \times 100$, and counts active support cases. The `action_view_*` methods return Odoo actions for total applications, stage queues, placed students, employers, internships, mentors, and the support queue.

`action_create_support_request()` opens a blank support-ticket form with the current user preselected as assignee.

6. Views, Actions, and Navigation

XML views are the user interface layer. They do not replace the model rules; they decide what users see and how they work with it. An `ir.actions.act_window` action opens a model in a particular view mode and can include a `domain` filter. A menu item points to an action.

File	What it defines	Why it matters
<code>views/menu_views.xml</code>	Root application menu and grouped Dashboard, Student Journey, Catalogue, Network, and Brand Content navigation.	Creates a work-oriented information architecture instead of a flat list of technical models.
<code>views/crm_lead_views.xml</code>	Lead form additions, Student Journey page, custom kanban, custom list, student and partner actions.	Gives counsellors the information needed to work a placement without leaving the opportunity.
<code>views/dashboard_views.xml</code>	Read-only management dashboard with KPI buttons, funnel, health, priority, support, and service panels.	Provides the operational overview and direct record access described in Figure 1.
<code>views/interntion_operations_views.xml</code>	Support ticket list/form, reference model views, operations tab on leads, legacy/action menu definitions.	Exposes escalation and operational detail in native Odoo interfaces.
Employer, internship, mentor, project, service, testimonial, FAQ view files	List/form/kanban layouts and actions for their corresponding models.	Each entity receives an interface appropriate to its work: browse catalogue, edit detail, or scan pipeline cards.

Why the kanban pipeline is used

The custom lead kanban groups records by `stage_id` and allows dragging between columns. The cards show the candidate/lead, stage, course, employer, mentor, and related details only when populated. Kanban is used because a placement team needs to see flow and stalled work, not merely a spreadsheet of rows. The tree/list view remains available for filtering, exporting, and dense comparison.

Important implementation detail: action context

Action contexts set defaults when a user creates a record from an action, for example `default_type: 'opportunity'` and `default_interntion_side: 'student'`. XML references such as `ref('crm_team_interntion')` must be resolved at module-load time through the XML `eval` attribute. Leaving `ref(...)` as client-evaluated text causes the browser error “Name ref is not defined”; the corrected operations actions now resolve IDs on the server.

7. Security and Access Control

Security is split into groups, record rules, and model access CSV rules. This separation is required because “can a user open this model?” and “which records can the user see?” are different questions.

Security layer	Where defined	Purpose
Groups	<code>security/interntion_security.xml</code>	Defines general users, managers, placement counsellors, partnership account managers, mentors, and operations administrators.
Record rules	<code>interntion_security.xml</code>	Restricts placement counsellors to Student Placements and partnership managers to Corporate Partnerships.
Model access	<code>security/ir.model.access.csv</code>	Defines read/write/create/delete permissions per model and role.

Managers receive broader CRUD rights because they own configuration and corrections. Regular staff can work operational data but cannot delete it, reducing accidental loss. Marketing-oriented data such as Services, Testimonials, and FAQs is read-only for regular users so approved public content stays controlled.

8. Automation, SLA, Support, and Sales

Hourly SLA monitor

`data/automation_data.xml` defines a scheduled job that calls `crm.lead.cron_monitor_slas()` every hour. The method searches non-won leads, identifies at-risk or breached cases, and schedules an activity only if an equivalent active escalation is not already present. The duplicate check matters: a recurring job must not create an unlimited number of identical alerts.

Email templates

The same data file defines templates for application receipt, partner acknowledgement, and placement completion. Templates are required for consistent, timely communication; they keep wording and recipient fields centralised rather than manually typed each time.

Support workflow

Request Help → Ticket linked to lead → Priority and assignee → New / In progress / Resolved / Closed → Chatter and activity history

The Help & Support dashboard panel is intentionally small. It does not try to duplicate a helpdesk product; it gives the operations team two immediate signals (open and urgent) and routes staff to the existing, auditable Odoo ticket form.

Add-on quotation workflow

When a student opportunity has selected `addon_service_ids`, the lead form exposes **Create Add-on Quotation**. The code creates a standard Odoo sales order and one line per add-on. This is required to connect

an operational CRM request to formal commercial records, pricing, and later invoicing.

9. Seed Data and Website Integration

The `data/` directory supplies initial records for services, stages, employers, mentors, internships, projects, testimonials, and FAQs. Seed data makes a new database useful immediately and keeps the CRM vocabulary aligned with Interntion's public offering. It is loaded through the manifest in a deliberate order: reference teams and stages first, catalogue records next, and views/actions afterward.

Controller endpoint	Result	Why the route exists
<code>/interntion/apply</code>	Creates a student-side CRM lead; accepts CV upload with a 5 MB limit.	Converts a public application into an operational case.
<code>/interntion/partner-enquiry</code>	Creates a partner-side lead in the corporate partnership team.	Keeps employer enquiries separate from student placement work from the first contact.
<code>/interntion/express-interest</code>	Creates a student interest lead.	Captures lower-friction interest before a complete application is submitted.

`controllers/website.py` uses `sudo()` when creating leads because website visitors are public users with no regular CRM permissions. It records UTM/source fields where supplied, enabling future marketing attribution. Because public routes accept input, production deployment should add rate limiting, CAPTCHA, and stricter upload scanning.

10. Operating the Project

Local commands

Task	Command	Reason
Start	<code>docker compose up -d --build</code>	Builds/starts the defined application services in the background.
Open CRM	<code>http://localhost:8069</code>	Odoo's published web port.
Validate Python	<code>docker compose exec -T odoo python3 -m py_compile /mnt/extra-addons/interntion_crm/models/dashboard.py</code>	Quickly catches Python syntax/indentation errors before module loading.
Apply module changes	<code>docker compose exec -T odoo odoo -c /etc/odoo/odoo.conf -u interntion_crm -d interntion_crm --stop-after-init</code>	Updates model metadata, data, views, and actions in the database.
Refresh running server	<code>docker compose restart odoo</code>	Ensures the long-running worker reloads the upgraded registry.

Recommended production controls

- Replace default database credentials and `admin_passwd`; never expose them in source control.
- Set `list_db = False` after database setup, place Odoo behind HTTPS reverse proxy, and set an appropriate worker count.
- Back up PostgreSQL regularly and test restoration, not merely backup creation.
- Use a recognised IANA timezone such as `Asia/Kolkata`, not obsolete aliases such as `Asia/Calcutta`, because PostgreSQL aggregation can reject unsupported zone names.
- For large data volumes, replace per-load dashboard computations with cached/reporting measures; the current dashboard intentionally calculates live counts and is best suited to normal operational volumes.

Current known scope boundaries

- `addon_revenue_month` currently displays `0.0`; it is a placeholder until revenue is derived from sales orders/invoices.
- Support escalation beyond ticket creation is operational/manual; only SLA lead escalation is automated.
- The dashboard depends on stage names as configured in seed data; future renaming should update both the stage configuration and dashboard mappings.

Summary: The module creates a connected operating system for the Interntion service journey: acquire an application or partner enquiry, qualify it, match and mentor the student, manage the internship, resolve support issues, close the placement, and analyse outcomes through live drill-down metrics.